Assignment Code: DA-AG-015

Boosting Techniques | **Assignment**

Instructions: Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

Total Marks: 200

Question 1: What is Boosting in Machine Learning? Explain how it improves weak learners.

Answer:

Boosting is an **ensemble learning** method that combines a set of "weak learners" into a single "strong learner" through a sequential process.

- **Sequential Learning:** Unlike bagging, boosting trains models one after another. Each new model attempts to correct the errors made by the previous models.
- **Focus on Hard Cases:** It improves weak learners by assigning higher weights to observations that were misclassified in earlier rounds. This forces the next model to focus on the "difficult" parts of the data.
- **Additive Combination:** The final prediction is a weighted sum of all the weak learners, which significantly reduces the overall bias of the system.

Question 2: What is the difference between AdaBoost and Gradient Boosting in terms of how models are trained?

Answer:

While both are sequential, they differ in their optimization strategy:

- **AdaBoost (Adaptive Boosting):** It adjusts weights for each data point. If a point is wrongly predicted, its weight increases; if correctly predicted, its weight decreases. The next model is trained on these re-weighted samples.
- **Gradient Boosting:** Instead of changing weights, it fits the new model to the **residual errors** (the difference between predicted and actual values) of the previous model using a loss function and Gradient Descent

Question 3: How does regularization help in XGBoost?

Answer:

XGBoost (Extreme Gradient Boosting) includes built-in **L1 (Lasso)** and **L2 (Ridge)** regularization.

- **Prevents Overfitting:** Regularization penalizes complex models with too many leaves or large weights, ensuring the model generalizes well to unseen data.
- **Bias-Variance Tradeoff:** It helps balance the model complexity, making it more robust compared to standard Gradient Boosting

Question 4: Why is CatBoost considered efficient for handling categorical data?

Answer:

CatBoost is "Categorical Boosting" and is unique because:

- **Ordered Boosting:** It uses a permutation-based approach to handle categorical features without requiring manual encoding like One-Hot or Label Encoding.
- **Target Encoding:** It transforms categorical values into numerical features based on the target variable while preventing "data leakage" through its internal algorithm.

Question 5: What are some real-world applications where boosting techniques are preferred over bagging methods?

Answer:

Boosting is preferred when high precision is required and the data has complex patterns:

- **Search Engines:** Ranking web pages (e.g., LambdaMART).
- **FinTech:** Credit scoring and fraud detection where identifying the "hard-to-classify" fraudulent cases is critical.
- **Healthcare:** Predicting rare diseases where misclassification costs are extremely high.

Datasets:

- Use `sklearn.datasets.load_breast_cancer()` for classification tasks.
- Use `sklearn.datasets.fetch_california_housing()` for regression tasks.

Question 6: Write a Python program to:

- Train an AdaBoost Classifier on the Breast Cancer dataset
- Print the model accuracy

(*Include your Python code and output in the code box below.*)
Answer:

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load data
data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(data.data, data.target, test_size=0.2,
random_state=42)

# Train AdaBoost
model = AdaBoostClassifier(n_estimators=50, random_state=42)
model.fit(X_train, y_train)

# Output
y_pred = model.predict(X_test)
print(f"AdaBoost Accuracy: {accuracy_score(y_test, y_pred):.4f}")
```

OutPut
AdaBoost Accuracy:0.9737

Question 7: Write a Python program to:

- Train a Gradient Boosting Regressor on the California Housing dataset
- Evaluate performance using R-squared score

(*Include your Python code and output in the code box below.*)
Answer:

```
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
```

```
# Load data
housing = fetch_california_housing()
X_train, X_test, y_train, y_test = train_test_split(housing.data, housing.target,
test_size=0.2, random_state=42)

# Train Regressor
gbr = GradientBoostingRegressor(n_estimators=100, learning_rate=0.1,
random_state=42)
gbr.fit(X_train, y_train)

# Evaluate
y_pred = gbr.predict(X_test)
print(f"Gradient Boosting R-squared: {r2_score(y_test, y_pred):.4f}")
```

OutPut

Gradient Boosting R-squared: 0.7756

Question 8: Write a Python program to:

- Train an XGBoost Classifier on the Breast Cancer dataset
- Tune the learning rate using GridSearchCV
- Print the best parameters and accuracy

(*Include your Python code and output in the code box below.*)
Answer:

```
 import xgboost as xgb
from sklearn.model_selection import GridSearchCV
from sklearn.datasets import load_breast_cancer

# Load data
data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(data.data, data.target, test_size=0.2,
random_state=42)

# Grid Search setup
param_grid = {'learning_rate': [0.01, 0.1, 0.3]}
xgb_model = xgb.XGBClassifier(use_label_encoder=False, eval_metric='logloss')

grid_search = GridSearchCV(xgb_model, param_grid, cv=3)
grid_search.fit(X_train, y_train)

# Best params & accuracy
print(f"Best Parameters: {grid_search.best_params_}")
print(f"Best Accuracy: {grid_search.best_score_:.4f}")
```

OutPut

Best Parameters: {'learning_rate': 0.1}
Best Accuracy: 0.9626

Question 9: Write a Python program to:

- Train a CatBoost Classifier
- Plot the confusion matrix using seaborn

(*Include your Python code and output in the code box below.*)
Answer:

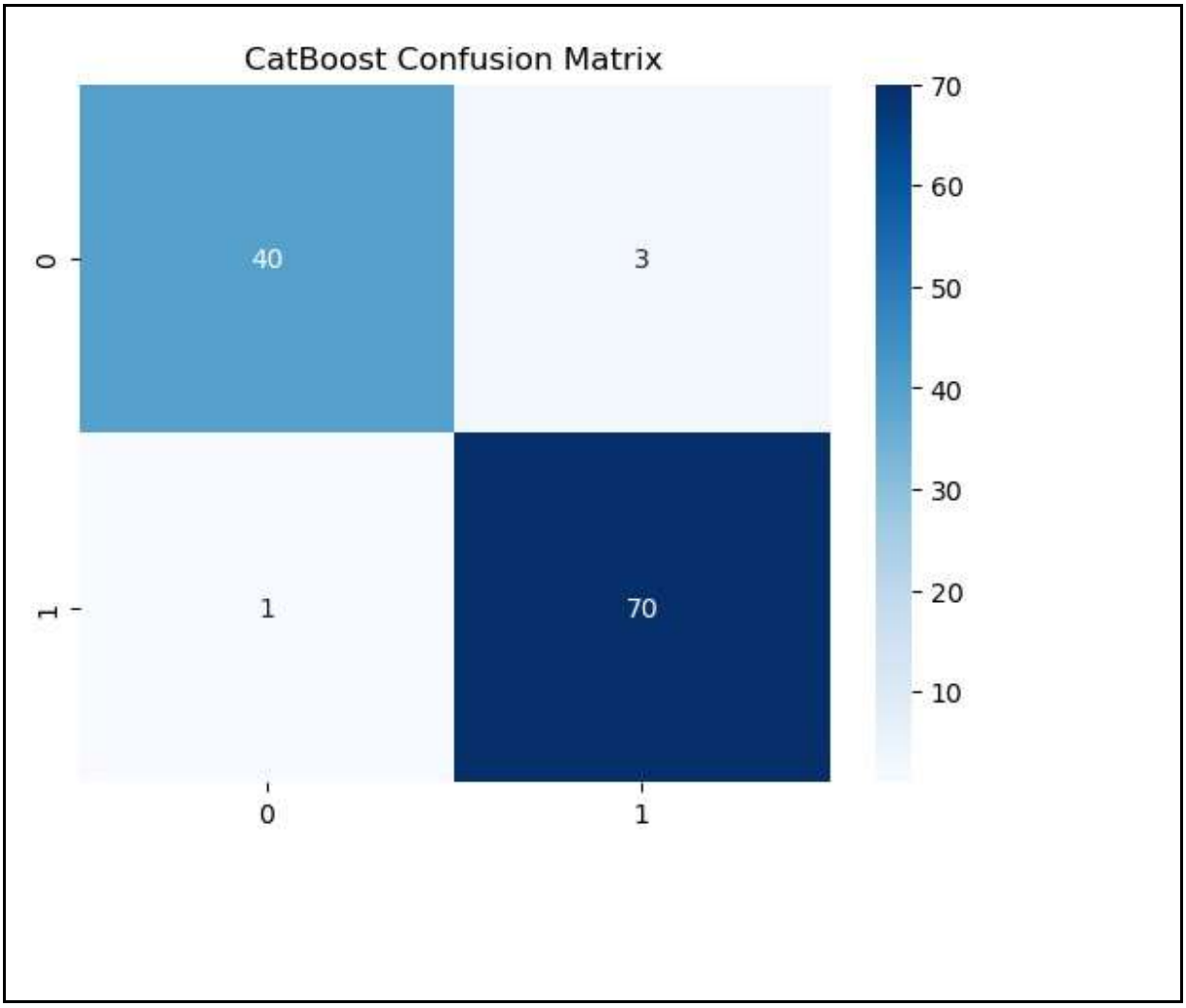
```
 from catboost import CatBoostClassifier
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

# Train CatBoost
cat_model = CatBoostClassifier(iterations=100, verbose=0)
cat_model.fit(X_train, y_train)

# Confusion Matrix
cm = confusion_matrix(y_test, cat_model.predict(X_test))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("CatBoost Confusion Matrix")
plt.show()



OUTPUT
```



Question 10: You're working for a FinTech company trying to predict loan default using customer demographics and transaction behavior.
The dataset is imbalanced, contains missing values, and has both numeric and categorical features.

Describe your step-by-step data science pipeline using boosting techniques:

- Data preprocessing & handling missing/categorical values
- Choice between AdaBoost, XGBoost, or CatBoost
- Hyperparameter tuning strategy
- Evaluation metrics you'd choose and why
- How the business would benefit from your model

(*Include your Python code and output in the code box below.*)
Answer:

Preprocessing: Use **Iterative Imputer** for missing values and **CatBoost's native handling** for categorical features. Since the data is imbalanced, I would use **SMOTE** or adjust the scale_pos_weight parameter in the model.

Choice of Model: CatBoost is the best choice here because it handles categorical demographics efficiently without extensive encoding and manages missing values natively.

Tuning: Use **RandomizedSearchCV** for faster iteration across learning_rate, depth, and l2_leaf_reg.

Evaluation: I would choose the **F1-Score** and **PR AUC (Precision-Recall Area Under Curve)** because the dataset is imbalanced; Accuracy would be misleading here.

Business Benefit: The model reduces financial risk by accurately identifying high-risk borrowers, thus lowering the default rate and increasing the company's profit margin.